RIVERTY · LEGAL TECH · AI CONTRACT REVIEW

Riverty Contract Review Complete Reference Guide — v2

Single source of truth: architecture, new retrieval stack, evaluation framework, Q&A

Prepared: 2026-04-02 | Branch: `main` | Build: 122 tests · 0 failures

RAG System · Hybrid BM25 + Dense · Cross-Encoder Reranking · 3 Query Modes · Azure-Ready · 22 Eval Metrics · 7 Categories · 122 Tests · 0 Failures

Table of Contents

1. What is this project? p.2

2. Problem Statement p.2

3. Solution Architecture (Overview) p.3

4. Full File & Folder Structure p.3

5. Tech Stack — Every Tool and WHY p.5

6. Data Flow — Step by Step p.6

7. Three Analysis Modes p.7

8. Demo vs Production — The Swap Pattern p.8

9. ETL Pipeline Deep Dive p.9

10. RAG Layer — Classic Retriever p.10

11. NEW: Hybrid Retriever — BM25 + Dense + RRF p.11

12. NEW: Cross-Encoder Reranker & MMR Filter p.12

13. LangGraph Agent — 4-Node Graph p.13

14. API Routes Reference p.14

15. Frontend Architecture p.15

16. How to Run the Project p.15

17. NEW: Evaluation Framework — 7 Categories, 22 Metrics p.16

18. Production Roadmap p.19

19. Interview Q&A — Every Question They Could Ask p.20

1. What is this project?

Riverty Contract Review is an AI-powered legal contract analysis system. It allows Riverty's legal team to:

- Upload contracts (PDF, scanned images) to a persistent knowledge base
- Ask plain-English questions: "Does this contract have a GDPR clause?"
- Compare a new contract against every existing contract in the knowledge base
- Detect missing or risky clauses across all stored contracts
- Get compliance check results in structured form (compliant / violations list / explanation)

The system runs in **demo mode** (on a laptop, no cloud required) and is architected to **swap to Azure production services** with minimal code change. Every demo component has a documented Azure equivalent.

122	22	7	1
TESTS · 0 FAILURES	EVAL METRICS	ADRS ON RECORD	ENV VAR TO PRODUCTION

2. Problem Statement

Riverty's legal team reviews hundreds of contracts annually. The manual review process has three pain points:

Pain Point	Impact
Time — reading full PDFs clause by clause	Hours per contract, delays business decisions
Inconsistency — different reviewers focus on different clauses	Compliance risks slip through
No cross-contract search — can't ask "which contracts have unlimited liability?"	Can't audit the full portfolio quickly

What the system solves:

- **Instant answers** — ask a plain-English question, get an answer with source citations in seconds
- **Compliance check** — automated structured check against GDPR, termination, liability, governing law
- **Portfolio analysis** — hybrid search (semantic + keyword) across all stored contracts to find patterns and gaps

3. Solution Architecture (Overview)

```
USER BROWSER
React + TypeScript SPA
┌──────────────────┬──────────────────┬──────────────────┐
│ Search KB        │ Analyze Doc      │ Compare with DB  │
│ (MODE 3)         │ (MODE 1)         │ (MODE 2)         │
└──────────┬───────┴──────────┬───────┴──────────┬───────┘
         │ SSE Stream        │ SSE Stream         │ SSE Stream
         ▼                   ▼                    ▼
┌──────────────────────────────────────────────────────┐
│                 FastAPI Backend :8000                 │
│  POST /api/query   POST /api/analyze   POST /api/ingest  │
│  POST /api/compliance   POST /api/eval/retrieve  [NEW]   │
├────────────────────┬─────────────────────────────────┤
│ ETL Pipeline       │ RAG Layer                       │
│ PDF/OCR →          │ EmbeddingService → HybridRetriever │
│ Cleaner →          │   BM25 + Dense + RRF  [NEW]     │
│ Chunker →          │ CrossEncoder Reranker [NEW]     │
│ Embedder →         │ MMR Filter            [NEW]     │
│ ChromaDB           │ LangGraph Agent (4 nodes)       │
└────────────────────┴─────────────────────────────────┘
         │
         ▼
ChromaDB (demo) ← swap → Azure AI Search (prod)
Local ./chroma_db/
```

4. Full File & Folder Structure

```
riverty-contract-review/
├── CLAUDE.md                          # AI coding rules and project conventions
├── .gitignore
│
├── backend/
│   ├── README.md                      # Tech stack, folder map, quick start
│   ├── SETUP.md                       # Step-by-step setup, Docker, troubleshooting
│   ├── Dockerfile                     # Python 3.12 image
│   ├── requirements.txt               # All Python dependencies pinned
│   ├── .env.example                   # Template for all env vars
│   ├── run_pipeline.sh                # Ingest documents via API + Q&A
│   ├── run_rag.sh                     # Q&A only (no ingest)
│   ├── run_tests.sh                   # pytest + coverage
│   ├── ingestion_registry.json        # MD5-based deduplication registry
│   │
│   └── app/
│       ├── __init__.py
│       ├── main.py                    # FastAPI app factory; registers all routers
│       ├── config.py                  # Pydantic Settings singleton — reads .env
│       ├── models.py                  # Shared Pydantic models (ComplianceResult, etc.)
│       ├── state.py                   # Shared app-level singletons (embedding model)
│       │
│       ├── api/
│       │   └── routes/
│       │       ├── health.py          # GET /health
│       │       ├── ingest.py          # POST /api/ingest (persistent KB)
│       │       ├── query.py           # POST /api/query (MODE 3 — SSE)
│       │       ├── analyze.py         # POST /api/analyze (MODE 1 & 2 — SSE)
│       │       ├── compliance.py      # POST /api/compliance (structured JSON)
│       │       ├── files.py           # GET /api/files/{filename}
│       │       ├── suggestions.py     # GET /api/suggested-questions
│       │       └── eval_retrieve.py   # POST /api/eval/retrieve [NEW — eval only]
│       │
│       ├── etl/
│       │   ├── base.py                # Abstract BaseExtractor, BaseTransformer
│       │   ├── pipeline.py            # IngestionPipeline orchestrator
│       │   ├── registry.py            # IngestionRegistry (MD5 dedup + model guard)
│       │   ├── compliance_storage.py  # Fire-and-forget external API archival
│       │   ├── extractors/
│       │   │   ├── pdf_extractor.py   # PyMuPDF page-by-page extraction
│       │   │   └── ocr_extractor.py   # Tesseract OCR for scanned images
│       │   ├── transformers/
│       │   │   ├── text_cleaner.py    # Whitespace, pipes, umlaut normalisation
│       │   │   ├── language_detector.py # langdetect → "en" / "de"
│       │   │   ├── content_type_detector.py  # qa / legal / narrative classifier
│       │   │   └── document_chunker.py # Three chunking strategies
│       │   └── loaders/
│       │       ├── base.py            # BaseLoader interface
│       │       ├── chroma_loader.py   # ChromaDB (demo) ↔ AzureSearchLoader swap
│       │       └── azure_search_loader.py  # Azure AI Search (production)
│       │
│       ├── rag/
│       │   ├── embeddings.py          # EmbeddingService (bge-m3 demo / Azure prod)
│       │   ├── retriever.py           # ContractRetriever — cosine similarity, threshold
│       │   ├── hybrid_retriever.py    # BM25 + dense + RRF merge [NEW]
│       │   ├── reranker.py            # CrossEncoder reranking + MMR filter [NEW]
│       │   ├── agent.py               # LangGraph Agent — 4-node graph
│       │   ├── llm_client.py          # Shared AsyncOpenAI / AsyncAzureOpenAI factory
│       │   ├── document_analyzer.py   # MODE 1 & 2 analysis functions
```

```
│       │   └── document_grouper.py     # Groups chunks by source_file for MODE 3
│       │
│       └── storage/
│           ├── local_storage.py        # Saves files to ./uploads/ (demo)
│           └── azure_blob_storage.py   # Azure Blob Storage (production)
│
├── frontend/
│   ├── package.json                    # React 18 + TypeScript + Vite + TailwindCSS
│   ├── vite.config.ts                  # Vite dev server proxies /api → localhost:8000
│   └── src/
│       ├── App.tsx                     # Root component — three-mode layout
│       ├── types/index.ts              # TypeScript interfaces
│       ├── services/
│       │   ├── api.ts                  # REST calls (ingest, compliance check, files)
│       │   └── streaming.ts            # SSE helpers — streamQuery, streamAnalyze
│       ├── hooks/
│       │   └── useDocumentAnalysis.ts  # File state + streaming + compliance state
│       └── components/
│           ├── Analysis/
│           │   ├── ModeSelector.tsx    # Three-tab mode switcher
│           │   ├── TemporaryUpload.tsx # Drop zone with "not stored" label
│           │   ├── CompliancePanel.tsx # Compliance result display
│           │   └── ExampleQueries.tsx  # Mode-specific query chips
│           ├── QueryInput.tsx          # Question input + submit
│           ├── StreamingResponse.tsx   # Token-by-token display
│           ├── SuggestedQueries.tsx    # Suggested query chips (MODE 3)
│           ├── FileUpload.tsx          # Persistent KB upload dropzone
│           └── IngestedFileList.tsx    # List of files in the knowledge base
│
├── docs/
│   ├── decisions.md                    # Architecture Decision Records (ADR-001 to ADR-007)
│   ├── implementation-status.md        # What's built, what's remaining
│   ├── env-reference.md               # Annotated .env template
│   ├── azure-services.md              # Six Azure services needed for production
│   ├── data-flow.md                   # Visual data flow diagram
│   └── setup.md                       # Mirror of SETUP.md
│
└── backend/tests/
    ├── unit/                           # 93 fast unit tests (offline, mocked)
    │   ├── test_pdf_extractor.py
    │   ├── test_chunker.py
    │   ├── test_hybrid_retriever.py    # [NEW]
    │   ├── test_reranker.py            # [NEW]
    │   └── ...
    └── eval/                           # [NEW] evaluation harness
        ├── run_eval.py                 # Main eval script — all 3 modes, 22 metrics
        ├── ground_truth.json           # Expected answers per question
        └── metrics/                    # Pluggable metric modules [NEW]
            ├── answer_metrics.py       # Faithfulness, answer relevance, completeness, hallucination
            ├── context_metrics.py      # Context precision, context recall
            ├── attribution_metrics.py  # Citation accuracy, citation coverage
            ├── cost_metrics.py         # Cost per query, session cost
            └── robustness_metrics.py   # Query consistency, noise sensitivity
```

5. Tech Stack — Every Tool and WHY

Backend

Tool	Version	What it does	Why chosen	Azure Prod Swap
FastAPI	0.111	HTTP API framework	Async-native; built-in OpenAI docs; `StreamingResponse` for SSE with one line	Azure API Management in front
LangGraph	1.1.3	AI agent orchestration	State-machine graph; each node independently testable and replaceable	Same — works with Azure OpenAI
LangChain	1.2.13	Text splitting utilities	`RecursiveCharacterTextSplitter` handles legal text better than simple split	Same
OpenAI GPT-4o	openai 2.30	LLM answer generation	Best legal reasoning; same SDK for Azure OpenAI swap	Azure OpenAI `gpt-4o` deployment
BAAI/bge-m3	sentence-transformers	Local embedding model	Free; offline; 1024-dim; multilingual (EN + DE); no API calls during ingest	Azure OpenAI `text-embedding-3-large` (3072-dim)
ChromaDB	1.5.5	Vector store	Zero-config; runs in-process; persists to disk; good enough for demo	Azure AI Search (hybrid vector + keyword)
rank_bm25	latest	BM25 keyword retrieval	In-memory BM25Okapi over ChromaDB chunks; handles exact legal terms (§12, "Riverty GmbH")	Azure AI Search native BM25 (built-in)
CrossEncoder (ms-marco-MiniLM)	sentence-transformers	Reranking	Reads query + chunk jointly; more accurate relevance than cosine bi-encoder similarity	Azure Cognitive Search semantic reranker
PyMuPDF	1.27	PDF text extraction	Fastest Python PDF lib; handles mixed-language PDFs; per-page extraction	Azure Document Intelligence
pytesseract	0.3.13	OCR for scanned images	Industry standard; handles German text; wraps Tesseract 5	Azure Document Intelligence
langdetect	1.0.9	Language detection	Lightweight; identifies German vs English; used to tag every chunk	Azure Cognitive Services Comprehend
Pydantic	2.12	Request/response validation	Type-safe config + models; `pydantic-settings` reads .env automatically	Same
httpx	async HTTP client	External compliance API calls	Async; timeout control; used for fire-and-forget compliance archival	Same

Frontend

Tool	What it does	Why chosen
React 18	UI framework	Industry standard; hooks-based; concurrent rendering for SSE
TypeScript	Type safety	Catches API shape mismatches at compile time
Vite	Build tool + dev server	10× faster than Webpack; native ESM; proxy config for API
TailwindCSS	Styling	Utility-first; no CSS files; consistent design tokens
Lucide React	Icons	Lightweight; consistent style; tree-shakeable

Why these specific choices matter for Riverty

1. **Azure-first architecture**: Every demo component has a documented Azure production swap. Riverty is a Microsoft shop — this alignment was a design constraint from day one.
2. **Offline embeddings**: `BAAI/bge-m3` runs locally so contract text never leaves the machine during the demo. In production, Azure OpenAI keeps data within the Azure tenant.
3. **German + English**: `bge-m3` is multilingual; `langdetect` tags every chunk; Tesseract is installed with German language data. Riverty operates in Germany.
4. **Source citations**: RAG (not fine-tuning) means every answer is traced to a specific document, page, and chunk — a legal requirement.
5. **Hybrid retrieval**: BM25 handles exact legal terms ("§12", entity names) that dense vectors miss. RRF merge requires no calibration between score scales.

6. Data Flow — Step by Step

Ingestion Flow (upload a contract to the persistent knowledge base)

```
User uploads contract.pdf via browser
        │
        ▼
POST /api/ingest (FastAPI)
        │
        ├── 1. Save raw file to ./uploads/ (LocalStorage)
        │
        ├── 2. IngestionRegistry.check(file)
        │       ├── Compute MD5 checksum of file bytes
        │       ├── If checksum already in registry.json → return status="skipped"
        │       └── If new → register checksum + model name + timestamp
        │
        ├── 3. ETL Pipeline.ingest(file_path)
        │       │
        │       ├── EXTRACT: PDFExtractor → list of {page_number, text} dicts
        │       │            (OCRExtractor for .jpg/.png)
        │       │
        │       ├── TRANSFORM:
        │       │   ├── TextCleaner → normalize whitespace, pipes, umlauts
        │       │   ├── LanguageDetector → tag "en" or "de"
        │       │   ├── ContentTypeDetector → classify as qa / legal / narrative
        │       │   └── DocumentChunker → split using appropriate strategy:
        │       │       ├── qa: split at Q: markers → one chunk per Q+A pair
        │       │       ├── legal: split at §, Article, Clause headers
        │       │       └── narrative: RecursiveCharacterTextSplitter (1500 chars, 200 overlap)
        │       │
        │       └── LOAD:
        │           ├── EmbeddingService.get_embeddings_batch(all chunk texts)
        │           │   → BAAI/bge-m3 produces 1024-dim vector per chunk
        │           └── ChromaLoader.load(chunks + embeddings)
        │               → Upsert into ChromaDB with 14 metadata fields
        │
        ├── 4. compliance_storage.store_contract_in_api() [fire-and-forget]
        │       → POST file bytes to COMPLIANCE_API_URL (if configured)
        │
        └── Return {"status": "indexed", "chunks_created": N, "language": "en"}
```

Query Flow — MODE 3 (cross-DB search) with Hybrid Retrieval NEW

```
User types: "Which contracts are missing a GDPR clause?"
        │
        ▼
POST /api/query {"question": "..."}
        │
        ▼
LangGraph Agent — 4-node graph:
        │
        ├── Node 1: query_router
        │   └── Classifies query into:
        │       find_clause / compare_contracts / summarise / general_question / find_missing
        │       (LLM call — fast, system prompt only)
        │
        ├── Node 2: retriever  [UPGRADED — now runs HybridRetriever]
        │   ├── BM25Okapi index built in-memory over ChromaDB corpus
        │   ├── Dense vector: EmbeddingService.get_embedding(question) → 1024-dim vector
        │   │                 ContractRetriever.retrieve(vector, top_k=16)
        │   │                 → ChromaDB cosine similarity search
        │   ├── BM25 keyword: BM25Okapi.get_scores(tokenized_query)
        │   │                 → exact term matching (§12, "Riverty GmbH")
        │   ├── RRF merge: score = Σ 1/(60 + rank_i) per list
        │   ├── CrossEncoder reranker: ms-marco-MiniLM-L-6-v2 (query, chunk) pairs
        │   └── MMR filter: diversity selection, λ=0.6 relevance / diversity balance
        │
        ├── Node 3: reasoner
        │   ├── If query_type == "find_missing":
        │   │   └── document_grouper.group_by_document(chunks)
        │   │       → Group chunks by source_file
        │   │       → Build per-document context sections
        │   │       → System prompt asks LLM to state verdict per document
        │   └── Else:
        │       └── Build flat context string from chunks
        │           System prompt: "Answer only from the provided contract excerpts"
        │           GPT-4o streaming → yields tokens
        │
        └── Node 4: formatter
            └── Appends source citations: "• contract.pdf — page 3, chunk 2/5 (0.87)"
                Streams final tokens with citations
```

7. Three Analysis Modes

Mode	Name	Data Source	Backend Route	When to use
MODE 1	Analyze Document	Uploaded file only — never stored	`POST /api/analyze` (mode=single)	"Review this new contract before signing"
MODE 2	Compare with DB	Uploaded file + ChromaDB	`POST /api/analyze` (mode=compare)	"How does this draft compare to our existing contracts?"
MODE 3	Search Contracts	ChromaDB only	`POST /api/query`	"Which of our stored contracts are missing GDPR clauses?"

MODE 1 — Single Document Analysis

- User uploads a PDF; text extracted to temp file, **immediately deleted** after extraction
- File content is *never* written to ChromaDB
- GPT-4o answers using only the uploaded document as context
- Also supports **compliance check** (`POST /api/compliance`) → returns structured JSON: `{"compliant": false, "violations": [...], "explanation": "..."}`

MODE 2 — Compare with Database

- User uploads a PDF (also never stored permanently)
- System embeds the first 500 chars as a semantic query
- Retrieves the most similar stored contracts from ChromaDB
- GPT-4o sees both the uploaded document AND the retrieved contracts
- Produces a comparison: "Your new contract differs from existing ones in these ways..."

MODE 3 — Cross-DB Query

- No file upload needed; user asks a question about the stored contract portfolio
- LangGraph's `query_router` classifies the query
- HybridRetriever (BM25 + dense + RRF) fetches relevant chunks NEW
- If `find_missing`: uses `document_grouper` to build per-document context → LLM gives per-document verdicts
- If `find_clause` / `compare_contracts` etc.: flat retrieval → standard grounded answer

8. Demo vs Production — The Swap Pattern

Every file that uses a demo tool has this comment block:

```
# ============================================================ # DEMO MODE: ChromaDB — zero-config vector store for laptop demo # PRODUCTION SWAP → Azure AI Search (AWS: OpenSearch): # Replace ChromaLoader() with AzureSearchLoader() in pipeline.py # Azure AI Search adds hybrid search, RBAC, and enterprise scale # ============================================================
```

The Seven Swaps

Layer	Demo	Production (Azure)	Where to change
Vector store	ChromaDB local disk	Azure AI Search	`etl/loaders/chroma_loader.py` → `AzureSearchLoader`
LLM	OpenAI `api.openai.com`	Azure OpenAI endpoint	`rag/llm_client.py` → `AsyncAzureOpenAI`
Embeddings	BAAI/bge-m3 local	Azure OpenAI `text-embedding-3-large`	`rag/embeddings.py`
BM25 index	rank_bm25 in-memory	Azure AI Search native BM25 (built-in, no separate index)	Remove `hybrid_retriever.py`, update `retriever.py`
File storage	Local `./uploads/`	Azure Blob Storage	`storage/local_storage.py` → `AzureBlobStorage`
PDF extraction	PyMuPDF + Tesseract	Azure Document Intelligence	`etl/extractors/`
Auth / identity	API key in `.env`	Azure Managed Identity	All Azure SDK clients

Why the swap is minimal

All swappable components are hidden behind abstract base classes (`BaseLoader`, `BaseExtractor`). The calling code (`pipeline.py`, `agent.py`) never imports the concrete class directly. The `APP_ENV` environment variable triggers the right branch:

```
# config.py
APP_ENV=development   # → demo stack
APP_ENV=production    # → Azure stack
```

> **Note on BM25 in production:** The in-memory `BM25Okapi` index (rank_bm25) is only needed in demo because ChromaDB doesn't support keyword search. Azure AI Search supports hybrid vector + BM25 natively in a single API call — `hybrid_retriever.py` can be deleted in production.

9. ETL Pipeline Deep Dive

Extractors (Strategy Pattern)

```
BaseExtractor (abstract)
    └── PDFExtractor   → PyMuPDF: page-by-page text, handles mixed EN/DE text
    └── OCRExtractor   → pytesseract: converts rasterized PDFs and images
```

The pipeline selects the extractor based on file extension: `.pdf` → `PDFExtractor` (if text is extractable) - `.jpg`, `.jpeg`, `.png` → `OCRExtractor` - Scanned PDFs → `OCRExtractor` (detected by empty text extraction)

Chunking Strategies

Three content types → three chunking strategies:

- **QA documents** (FAQ contracts, questionnaires): Split at `Q:` markers. One chunk = one question + answer pair. Preserves semantic unity of each Q&A.
- **Legal documents** (contracts, agreements): Split at `§`, `Article N`, `Clause N`, `SECTION N` headers. One chunk = one legal clause/article. Preserves the atomic unit of legal meaning.
- **Narrative documents** (everything else): `RecursiveCharacterTextSplitter(chunk_size=1500, chunk_overlap=200)`. Tries `\n\n` → `\n` → `.` before hard character split. 200-char overlap prevents sentences being cut at boundaries.

Chunk Metadata (14 fields)

Every stored chunk carries:

```
{
    "source_file": "contract_2024.pdf",
    "page_number": 3,
    "chunk_index": 2,
    "total_chunks": 8,
    "language": "en",            # or "de"
    "content_type": "legal",     # qa / legal / narrative
    "checksum": "a3f2...",       # MD5 of original file
    "file_size_kb": 142,
    "extraction_method": "pdf",  # or "ocr"
    "is_scanned": False,
    "embedding_model": "BAAI/bge-m3",
    "upload_timestamp": "2026-03-30T14:22:00Z",
    "text_preview": "The termination notice period...",
    "chunking_strategy": "legal"
}
```

Deduplication Registry

`ingestion_registry.json` tracks every ingested file:

- **MD5 checksum** — same file → `status="skipped"`, no re-embedding
- **Embedding model name** — if model changed → `409 Conflict` (different dims, cannot mix)

10. RAG Layer — Classic Retriever

EmbeddingService

```
# Demo: local, free, offline, 1024-dim, multilingual
model = SentenceTransformer("BAAI/bge-m3")

# Production swap: Azure OpenAI, 3072-dim, within Azure tenant
client = AzureOpenAI(...)
client.embeddings.create(model="text-embedding-3-large", input=texts)
```

Supports: `get_embedding(text)` → single 1024-dim vector and `get_embeddings_batch(texts)` → efficient bulk embedding for ingestion.

ContractRetriever (classic — cosine similarity only)

```
results = collection.query(
    query_embeddings=[query_vector],
    n_results=top_k,
    include=["documents", "metadatas", "distances"]
)
# Filter: keep only results with similarity_score >= 0.40
```

The 0.40 threshold was calibrated for `BAAI/bge-m3` normalized vectors. For Azure `text-embedding-3-large`, lower to ~0.30 (OpenAI models produce lower raw cosine scores). The classic retriever is still used in compliance checks and analyze routes.

11. NEW: Hybrid Retriever — BM25 + Dense + RRF NEW

> **Why hybrid matters for legal contracts:** Dense vectors handle paraphrased queries ("ending the agreement" ↔ "termination"). BM25 handles exact legal terms that dense search can miss — "Riverty GmbH", "§12", "Article 17 GDPR". RRF merges both without requiring comparable score scales.

```
# ============================================================ # DEMO MODE: BM25 index built in-memory over ChromaDB chunk texts. # PRODUCTION SWAP → Azure AI Search hybrid mode (AWS: OpenSearch hybrid): # Azure AI Search runs BM25 + vector natively in a single API call. # No separate BM25 index needed — remove this file and update retriever.py. # ============================================================
```

Architecture

```
class HybridRetriever:
    # In-memory BM25Okapi index — rebuilt lazily when collection count changes
    # ChromaDB cosine similarity for dense search
    # RRF_K = 60 (standard constant)

    def retrieve(query, top_k=8):
        candidates = top_k * 2  # cast wide net before merging

        dense_results  = _dense_retrieve(query, candidates)
        bm25_results   = _bm25_retrieve(query, candidates)
        merged_results = _rrf_merge(dense_results, bm25_results, top_k)
        return merged_results
```

Reciprocal Rank Fusion (RRF)

RRF combines two ranked lists without needing calibrated scores between them:

```
RRF score for chunk C = Σ over each list L of: 1 / (K + rank_of_C_in_L)

Example (K=60):
  Chunk "§12 Termination" ranked 1st in BM25, 3rd in dense:
    RRF = 1/(60+1) + 1/(60+3) = 0.01639 + 0.01587 = 0.03226

  Chunk "cancellation clause" ranked 2nd in dense only:
    RRF = 1/(60+2) = 0.01613

  → "§12 Termination" wins because it appears in both lists.
```

BM25 Index Management

- The BM25 index is built **lazily** on first query and cached in memory
- Rebuilt automatically when ChromaDB collection count changes (new ingest detected)
- Tokenization: simple whitespace split (lowercase) — sufficient for legal clause matching
- Empty collection: BM25 returns empty, system falls back gracefully to dense-only

Fallback logic

- If BM25 returns nothing → return dense results only (top_k)
- If dense returns nothing → return BM25 results only (top_k)
- If both return results → full RRF merge

12. NEW: Cross-Encoder Reranker & MMR Filter NEW

Cross-Encoder Reranking

After HybridRetriever returns candidate chunks, the cross-encoder reranker improves precision:

```
Model: cross-encoder/ms-marco-MiniLM-L-6-v2  (~80 MB, cached after first download)

# Bi-encoder (used in retrieval):
  query_embedding = embed(query)        # independent
  doc_embedding   = embed(chunk_text)   # independent
  score = cosine(query_embedding, doc_embedding)

# Cross-encoder (used in reranking):
  score = model.predict([(query, chunk_text)])  # joint scoring
  # Reads both texts together — much more accurate relevance
```

Why it matters: Short legal clauses often share surface form with many queries. A bi-encoder may score "The agreement may be terminated..." highly for both "termination notice period" and "payment terms" because both mention agreements. A cross-encoder reading them jointly correctly distinguishes them.

Fallback: If the cross-encoder model cannot be loaded (e.g. no internet on first download), the system falls back silently to sorting by `similarity_score` — the retrieval pipeline continues to work.

Maximal Marginal Relevance (MMR) Filter

After reranking, MMR selects a diverse subset of chunks to prevent the LLM seeing the same clause repeated 8 times:

```
MMR score for candidate C = λ × relevance(C) - (1-λ) × max_similarity(C, selected)

Parameters:
  λ = 0.6  →  60% relevance weight, 40% diversity penalty
  Inter-chunk similarity: Jaccard overlap of token sets (no extra model needed)

Algorithm: greedy iterative selection
  1. Start with empty selected set
  2. Pick chunk with highest MMR score from remaining candidates
  3. Add to selected, repeat until top_k chunks selected
```

Result: the LLM receives a diverse set of chunks covering different contracts and different aspects of the query, improving answer quality and reducing redundant context.

Full Retrieval Pipeline (Mode 3 query)

```
Query
  │
  ▼
HybridRetriever.retrieve(query, top_k=16)
  ├── BM25Okapi.get_scores(tokenized_query) → top-16 keyword matches
  ├── ChromaDB cosine search → top-16 dense matches
  └── RRF merge → top-16 combined candidates
  │
  ▼
CrossEncoder.rerank(query, candidates, top_k=8)
  → (query, chunk_text) joint scoring → reordered top-8
  │
  ▼
mmr_filter(reranked, top_k=8, λ=0.6)
  → diverse selection → final 8 chunks
  │
  ▼
LangGraph reasoner node → GPT-4o
```

13. LangGraph Agent — 4-Node Graph

```
State: {question, query_type, retrieved_chunks, answer}

query_router → retriever → reasoner → formatter
     |              |           |           |
 classifies    HybridRetriever  GPT-4o   add source
 query type    + reranker     streaming   citations
               + MMR
```

Node 1 — query_router: Classifies the question into one of: `find_clause`, `compare_contracts`, `summarise`, `general_question`, `find_missing`. This changes how the context is built in the reasoner.

Node 2 — retriever: Runs HybridRetriever → CrossEncoder → MMR (as described in Section 12). Returns top-8 diverse, reranked chunks with all metadata.

Node 3 — reasoner:

- If `find_missing`: groups chunks by document, builds per-document context, uses a system prompt that asks GPT-4o to state per-document verdicts
- Otherwise: builds flat context string, uses standard grounded-answer system prompt

Node 4 — formatter: Appends clean source citations with relevance dots:

- ● = 0.65–1.0 (strongly relevant)
- ◖ = 0.40–0.65 (relevant)
- ○ = below threshold (filtered before reaching formatter)

Streaming

All three modes use Server-Sent Events (SSE):

```
POST /api/query or /api/analyze
→ StreamingResponse(media_type="text/event-stream")
→ Each token: "data: token\n\n"
→ End: "data: [DONE]\n\n"
```

React reads with `fetch()` + `response.body.getReader()` (not `EventSource`) — allows POST with body.

14. API Routes Reference

Method	Path	Auth	Request	Response	Mode
GET	`/health`	None	—	`{status, mode, document_count, version}`	—
POST	`/api/ingest`	None	`multipart: file`	`{status, chunks_created, language, ...}`	—
POST	`/api/ingest-all`	None	—	`{results: [...]}`	—
POST	`/api/query`	None	`{"question": str}`	SSE stream	MODE 3
POST	`/api/analyze`	None	`multipart: file, question, mode`	SSE stream	MODE 1/2
POST	`/api/compliance`	None	`multipart: file, guidelines?`	`ComplianceResult` JSON	MODE 1
GET	`/api/files/{filename}`	None	—	PDF bytes	—
GET	`/api/suggested-questions`	None	—	`[{id, text, category}]`	—
POST	`/api/eval/retrieve`	None	`{"question": str, "top_k": int}`	`{question, chunks[], total_retrieved}`	Eval only NEW
GET	`/docs`	None	—	Swagger UI	—

ComplianceResult model

```
class ComplianceResult(BaseModel):
    compliant: bool
    violations: list[str]   # empty if compliant
    explanation: str        # human-readable assessment
```

Default compliance guidelines checked: GDPR / data protection clause · Termination rights and notice period · Governing law and jurisdiction · Liability cap and limitation clauses · Clear party identification

POST /api/eval/retrieve — Evaluation Endpoint NEW

Returns raw retrieved chunks for a query without generating an LLM answer. Used exclusively by the evaluation harness (`tests/eval/run_eval.py`) to compute IR metrics.

Runs the same pipeline as the production query agent: HybridRetriever → CrossEncoder reranker → MMR filter. This ensures eval metrics reflect real retrieval performance.

15. Frontend Architecture

Three-Mode Layout

```
Left Sidebar              Right Panel
_________________         ________________________________
Knowledge Base            [Search] [Analyze] [Compare] ← ModeSelector
______________            |
FileUpload                |  MODE 3 (Search):
(persistent,              |    SuggestedQueries
 stored in DB)            |    QueryInput
                          |    StreamingResponse
IngestedFileList          |
                          |  MODE 1 (Analyze):
                          |    TemporaryUpload
                          |    ExampleQueries
                          |    QueryInput
                          |    [Check Compliance] button
                          |    CompliancePanel
                          |    StreamingResponse
                          |
                          |  MODE 2 (Compare):
                          |    TemporaryUpload
                          |    ExampleQueries
                          |    QueryInput
                          |    StreamingResponse
```

Key Design Decisions

- **Temporary uploads never reach ChromaDB** — enforced at the route level. The `TemporaryUpload` component shows a blue "Temporary" badge and "not stored in knowledge base".
- **useDocumentAnalysis hook** manages: `analysisFile`, streaming state (`isStreaming`, `content`, `isDone`), compliance state (`complianceResult`, `isCheckingCompliance`)
- **Mode switching** clears all response state — prevents stale SSE content from one mode appearing in another.
- **SSE reading** uses a shared `readSseStream()` helper in `streaming.ts`.

16. How to Run the Project

First-time Setup (5 minutes)

```
# 1. Clone
git clone <repo-url>
cd riverty-contract-review

# 2. Backend setup
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 3. Configure
cp .env.example .env
# Edit .env — set OPENAI_API_KEY=sk-...

# 4. (Optional) Pre-download embedding model
python -c "from sentence_transformers import SentenceTransformer; SentenceTransformer('BAAI/bge-m3')"
```

Start the Backend

```
cd backend
source .venv/bin/activate
uvicorn app.main:app --reload --port 8000
# → http://localhost:8000/docs  (Swagger UI)
# → http://localhost:8000/health
```

Start the Frontend

```
cd frontend
npm install
npm run dev
# → http://localhost:5173
```

Add Contracts to the Knowledge Base

```
# Option A: via UI — drag PDF to left sidebar dropzone
# Option B: via API
curl -X POST http://localhost:8000/api/ingest -F "file=@contract.pdf"

# Option C: batch ingest all files in uploads/
bash backend/run_pipeline.sh --ingest-only
```

Run All Tests

```
cd backend
source .venv/bin/activate
bash run_tests.sh
# → 122 tests, 0 failed
```

Run the Evaluation Harness NEW

```
cd backend
source .venv/bin/activate
# Start the server first, then:
python tests/eval/run_eval.py \
    --base-url http://localhost:8000 \
    --modes 1,2,3 \
    --top-k 8 \
    --output-dir results/

# Options:
#   --skip-ingest     skip re-ingestion if contracts already indexed
#   --modes 1,2,3     which modes to evaluate (default: all)
#   --top-k 8         retrieval top-k (default: 8)
# Results saved to: results/eval_YYYYMMDD_HHMMSS.json
```

> **Eval requires OPENAI_API_KEY:** Faithfulness scoring uses GPT-4o-mini as an LLM judge. If the key is absent, the system automatically falls back to local embedding cosine similarity as a faithfulness proxy — no error, just lower accuracy on that metric.

17. NEW: Evaluation Framework — 7 Categories, 22 Metrics NEW

Why domain-specific metrics matter: Generic NLP metrics (BLEU, ROUGE) measure word overlap. They don't tell you whether the system correctly identified a missing GDPR clause. *Contract Hit, Keyword Hit, and Absent Keyword Pass* measure exactly that — and can catch failures that BLEU would score as "passing".

Overview — All 22 Metrics

1. Retrieval (3 metrics)

- **Precision@K (P@K)** — fraction of retrieved chunks that are relevant
- **Recall@K (R@K)** — fraction of relevant chunks that were retrieved
- **Mean Reciprocal Rank (MRR)** — average of 1/rank of first relevant result

2. Generation (4 metrics)

- **Faithfulness (Faith[src])** — how well answer stays grounded in retrieved chunks
- **Answer Relevance (Ans-Rel)** — does the answer actually address the question?
- **Completeness Score (Complete)** — does the answer cover all expected points?
- **Hallucination Rate (Halluc)** — fraction of claims not supported by retrieved context

3. Task-Specific / Legal Domain (4 metrics)

- **Contract Hit (C-Hit)** — Mode 2: does answer reference the correct contract? (binary)
- **Keyword Hit (Kw-Hit)** — Modes 1,2,3: do expected keywords appear in the answer?
- **Absent Keyword Pass (AbsKw)** — Mode 1: does the system correctly say a keyword is absent?
- **Comparison Hit (Cmp-Hit)** — Mode 3: does the comparison answer both contracts correctly?

5. Attribution (2 metrics)

- **Citation Accuracy (Cit-Acc)** — are cited source files actually real files in the KB?
- **Citation Coverage (Cit-Cov)** — what fraction of the answer's claims have citations?

6. Robustness (2 metrics)

- **Query Consistency (Consist)** — does system give consistent answers to paraphrased queries?
- **Noise Sensitivity (NoiseSens)** — how much does answer change when noise chunks are injected?

7. System Performance (2 metrics)

- **Latency (Lat)** — wall-clock time from request to final SSE token (ms)
- **Cost per Query (Cost)** — estimated OpenAI token cost per query (USD)

4. Context Quality (2 metrics)

- **Context Precision (Ctx-Prec)** — are retrieved chunks actually useful to GPT-4o?
- **Context Recall (Ctx-Rec)** — did we miss any relevant chunks that should have been retrieved?

Metric Implementations — Pluggable Module Architecture

```
backend/tests/eval/
├── run_eval.py          # Orchestrator — runs all three modes, collects all metrics
├── ground_truth.json    # Expected answers per test case
└── metrics/             # Pluggable metric modules
    ├── answer_metrics.py       # Faithfulness, Answer Relevance, Completeness, Hallucination
    ├── context_metrics.py      # Context Precision, Context Recall
    ├── attribution_metrics.py  # Citation Accuracy, Citation Coverage
    ├── cost_metrics.py         # Cost per Query, Session Cost aggregation
    └── robustness_metrics.py   # Query Consistency, Noise Sensitivity + paraphrase generator
```

Category 1: Retrieval Metrics (IR metrics)

Computed via `POST /api/eval/retrieve` — returns raw chunks without LLM answer generation:

Metric	Formula	Requires
Precision@K	|relevant ∩ retrieved| / K	Ground-truth relevant chunk IDs per question
Recall@K	|relevant ∩ retrieved| / |relevant|	Ground-truth relevant chunk IDs per question
MRR	mean over queries of 1/rank(first relevant chunk)	Ground-truth relevant chunk IDs per question

Category 2: Generation Metrics

Faithfulness — two backends (automatic fallback)

1. **LLM judge (primary)**: GPT-4o-mini is given the question, retrieved context, and the system's answer. It scores faithfulness 0–10 with a structured prompt asking: "Does every factual claim in the answer appear in the provided context?" Score normalised to 0–1.
2. **Local embedding cosine (fallback)**: If `OPENAI_API_KEY` is absent, computes mean cosine similarity between context chunk embeddings and the answer embedding using `BAAI/bge-m3`. Always available offline.

Answer Relevance

GPT-4o-mini generates N back-questions from the system's answer, then measures cosine similarity between each back-question embedding and the original question embedding. High similarity → answer actually addresses the question asked.

Completeness Score

For each expected keyword/concept in the ground truth, checks whether it appears in the system's answer (case-insensitive, with synonym matching). Completeness = matched_concepts / total_expected_concepts.

Hallucination Rate

Inverse of faithfulness — fraction of claims in the answer that cannot be traced to any retrieved chunk. Uses the same LLM judge or embedding fallback as faithfulness.

Category 3: Legal Domain Metrics

Metric	Mode	How computed
Contract Hit (C-Hit)	2	Binary: does the answer name the correct comparison contract(s) from ground truth?
Keyword Hit (Kw-Hit)	1, 2, 3	Fraction of expected legal keywords present in answer (e.g. "GDPR", "termination", "§12")
Absent Keyword Pass (AbsKw)	1	For questions where a clause is expected to be absent, does the system correctly say "not found"?
Comparison Hit (Cmp-Hit)	3	For "which contracts differ in X" queries — does the answer correctly identify both contracts?

Category 4: Context Quality Metrics

These measure the quality of retrieved context, not the generated answer:

Metric	How computed
Context Precision	Fraction of retrieved chunks that GPT-4o-mini judges as "directly useful" for answering the question. High precision means the retriever wastes fewer LLM context window tokens on irrelevant material.
Context Recall	Fraction of ground-truth relevant information that appears in the retrieved context. Low recall means the system is missing crucial clauses even though it might answer partially correctly.

Category 5: Attribution Metrics

Metric	How computed
Citation Accuracy (Cit-Acc)	Parses source citations from the answer (filename · page · chunk format). Checks each cited source_file exists in the ingested KB. Score = valid_citations / total_citations.
Citation Coverage (Cit-Cov)	Ratio of answer sentences that have at least one citation. High coverage = every claim is traceable. Critical for legal audit requirements.

Category 6: Robustness Metrics

Query Consistency (Consist)

The `generate_paraphrases()` function creates N semantically-equivalent versions of each test question. The same question is then sent as each paraphrase. Consistency = similarity between all N answers (measured by

pairwise cosine similarity of answer embeddings). A low score means the system gives inconsistent answers to the same question worded differently.

Noise Sensitivity (NoiseSens)

The `inject_noise_chunks()` function appends irrelevant distractor chunks to the retrieved context before passing to the LLM. The `score_noise_sensitivity()` function measures how much the answer changes relative to the clean-context answer. Low sensitivity = robust to irrelevant context.

Category 7: System Performance

Metric	How measured
Latency (Lat)	Wall-clock time from POST request sent to final SSE `[DONE]` token received. Measured with `time.perf_counter()`. Includes all retrieval, reranking, LLM generation, and SSE streaming time.
Cost per Query (Cost)	`estimate_query_cost()` in `cost_metrics.py` estimates input/output token counts and multiplies by current GPT-4o pricing. `aggregate_session_cost()` sums across all eval queries.

Evaluation Output Format

```
# results/eval_20260402_143022.json
{
    "run_timestamp": "2026-04-02T14:30:22",
    "model": "gpt-4o",
    "embedding_model": "BAAI/bge-m3",
    "top_k": 8,
    "summary": {
        "mode_1": {"contract_hit": 0.92, "keyword_hit": 0.88, "faithfulness": 0.91,
                   "avg_latency_s": 2.3, "avg_cost_usd": 0.004},
        "mode_2": {"precision_at_k": 0.75, "recall_at_k": 0.68, "mrr": 0.82,
                   "contract_hit": 0.85, "faithfulness": 0.89, "avg_latency_s": 3.1},
        "mode_3": {"keyword_hit": 0.86, "comparison_hit": 0.80, "faithfulness": 0.87,
                   "context_precision": 0.79, "citation_accuracy": 0.98,
                   "query_consistency": 0.84, "noise_sensitivity": 0.12}
    },
    "per_question_results": [ ... ]
}
```

18. Production Roadmap

NOW — READY TODAY

- Full ETL pipeline (PDF + OCR)
- LangGraph agent — 4 nodes
- 3 query modes with SSE streaming
- React frontend — drag-drop, citations
- Hybrid retrieval: BM25 + dense + RRF
- Cross-encoder reranking + MMR filter
- Evaluation framework: 7 categories, 22 metrics
- 122 tests, 7 ADRs
- Azure swap pattern in every file

SHORT-TERM (1–2 WEEKS)

- `GET /api/files` listing — sidebar survives page refresh
- Re-index ChromaDB with consistent 1500-char chunks
- Stream compliance check (currently waits full response)
- Update ADR-005 to reflect 1500-char chunk size
- `VITE_` prefix for `BASE_URL` in frontend .env

PRODUCTION SWITCH

- Set `APP_ENV=production` + 6 Azure credentials
- Deploy to Azure Container Apps
- Azure AD SSO — no separate login
- CORS update for production domain
- Terraform modules for all Azure resources (~2–3 days)
- Remove `hybrid_retriever.py` — Azure AI Search handles BM25 natively

ADVANCED ENHANCEMENTS

- Redis cache for embeddings + frequent queries
- Parallel BM25 + dense search calls to reduce latency
- Azure Service Bus for async ingestion at scale
- Contract type classification (NDA / Service / Vendor)
- Multi-file batch upload (zip ingest)
- SharePoint integration — auto-ingest on upload
- RAGAS integration for automated faithfulness scoring

6 Azure credentials needed for production switch:

`AZURE_OPENAI_API_KEY` · `AZURE_SEARCH_KEY` · `AZURE_BLOB_CONN_STR` · `AZURE_DOC_INTEL_KEY` · `AZURE_TENANT_ID` · `AZURE_CLIENT_ID`

19. Interview Q&A — Every Question They Could Ask

Architecture & Design

Q: Why RAG instead of fine-tuning an LLM on contract data?

A: Three reasons. First, contracts change frequently — a new contract is queryable seconds after upload; fine-tuning requires a full retraining cycle. Second, legal requirement: every answer must cite the source clause — RAG gives this automatically through chunk metadata; fine-tuning cannot. Third, GDPR: storing contract content in model weights is hard to audit and impossible to delete (right to erasure requires deletion from the vector store entry, not retraining the model).

Q: Why LangGraph instead of a simple LLM call?

A: LangGraph gives us a typed state machine. Each of the four nodes (router, retriever, reasoner, formatter) is independently testable, replaceable, and observable. The query router node means we handle different query types differently — "find missing clauses" uses document-grouped context; "find a specific clause" uses flat retrieval. LangGraph also enables conditional branching and future additions (e.g., a reranker node, a hallucination check node) without refactoring the whole pipeline.

Q: Why Server-Sent Events instead of WebSocket?

A: SSE is strictly server-to-client, which perfectly matches our use case — the client sends one question and only needs to receive a streamed answer. SSE works through all HTTP proxies and corporate firewalls as plain HTTP with no protocol upgrade. WebSocket adds bidirectional complexity (ping/pong, reconnect logic) with zero benefit here. FastAPI's `StreamingResponse` supports SSE natively.

Q: Why ChromaDB for demo and Azure AI Search for production?

A: ChromaDB requires zero configuration — no cloud account, no API keys beyond OpenAI, no separate service to start. It runs in-process with Python and persists to local disk. A reviewer can clone the repo, set one API key, and have the full system running in 5 minutes. Azure AI Search adds what ChromaDB lacks in production: hybrid vector + keyword search (better recall on short legal terms), enterprise RBAC, horizontal scaling across pods, and SOC 2 / ISO 27001 compliance. The swap is one line: `ChromaLoader()` → `AzureSearchLoader()`.

Q: Why add BM25? Isn't dense vector search enough? NEW

A: Dense vectors are excellent at paraphrase matching ("ending the agreement" ↔ "termination") but weak on exact term matching. Legal contracts contain critical exact terms: specific article numbers (§12), entity names ("Riverty GmbH"), regulation citations ("Article 17 GDPR"). A dense model may embed "§12" similarly to many other paragraph references, losing the specificity. BM25 gives exact keyword matching. RRF merges both without requiring calibrated score scales — a chunk that appears in both ranked lists scores higher than one that appears in only one. In production, Azure AI Search provides this hybrid natively, so no additional code is needed.

Q: Why cross-encoder reranking? NEW

A: Bi-encoders (used in retrieval) encode the query and each chunk independently, then compare embeddings. This is fast but misses fine-grained relevance signals. A cross-encoder reads the query and chunk text jointly — it can catch that "the agreement terminates" is highly relevant to "termination notice period" but not to "payment

terms", even though both contain "agreement". The model used (ms-marco-MiniLM-L-6-v2) is ~80MB and runs on CPU. The cross-encoder scores override the retrieval scores, improving precision at the top of the final ranked list presented to GPT-4o.

Q: Why MMR diversity filtering? NEW

A: Without diversity filtering, the top-8 retrieved chunks might all be slight variations of the same termination clause from the same document — wasting the LLM's context window and producing a narrow, repetitive answer. Maximal Marginal Relevance iteratively selects the next chunk that maximises relevance while penalising similarity to already-selected chunks. The λ=0.6 parameter keeps 60% relevance weight — we prioritise quality but actively remove redundancy. This means GPT-4o sees diverse evidence from different contracts and different clause types.

Q: Why BAAI/bge-m3 instead of OpenAI text-embedding-3-large for the demo?

A: Three reasons. First, cost: bge-m3 is free — no API calls during ingestion, which is the most expensive phase. Second, privacy: contract text never leaves the machine during embedding — only GPT-4o queries hit an external API. Third, multilingual: bge-m3 handles German and English in one model without switching. In production we switch to Azure text-embedding-3-large because Azure OpenAI keeps data within the Microsoft tenant, which is required for Riverty's real contracts.

Q: How does the system prevent hallucination?

A: The reasoner node's system prompt explicitly instructs GPT-4o: "Answer only from the provided contract excerpts. If the information is not in the excerpts, say so." The formatter node appends source citations with relevance scores — users can verify every claim against the original document. The similarity threshold (0.40) filters out low-relevance chunks before they reach GPT-4o, so the model sees only genuinely relevant context. We do not prevent all hallucination (no system does) but we make it auditable.

Evaluation Framework NEW

Q: Why 22 metrics? Isn't accuracy enough?

A: "Accuracy" is undefined for open-ended RAG systems. The 22 metrics cover different failure modes that single-number accuracy cannot capture: a system can score well on BLEU but still fail to identify a missing GDPR clause (legal domain metric failure). It can retrieve relevant chunks but present them in a confusing order (context quality metric failure). It can give correct answers but without citations (attribution metric failure). Each category catches a different kind of failure. In practice, for a production deployment we'd focus on 3–5 key metrics per mode, but having all 22 defined lets us diagnose root causes precisely.

Q: What is faithfulness and how do you measure it without RAGAS?

A: Faithfulness measures whether the generated answer is grounded in the retrieved context — i.e., no claims beyond what the context provides. We use a two-tier approach: (1) GPT-4o-mini as an LLM judge — it reads the question, context, and answer and scores on a 0–10 scale how well the answer stays within the provided excerpts. This is the most accurate method. (2) Local embedding cosine similarity — if OpenAI API is unavailable, we compute mean cosine similarity between context chunk embeddings and the answer embedding using bge-m3. This is an offline proxy. RAGAS is the production-grade path for both faithfulness and answer relevancy — integrating it is a one-sprint task.

Q: How does query consistency testing work?

A: The `generate_paraphrases()` function creates semantically-equivalent question variants. For "Does contract A have a GDPR clause?" it might generate "Is there a data protection provision in contract A?" and "Does contract A address GDPR requirements?". We send all variants to the system and compare answers using pairwise cosine similarity of answer embeddings. A high consistency score (≥0.85) means the system gives the same answer regardless of phrasing. This is particularly important for legal use cases where different reviewers may phrase the same question differently.

Production & Scaling

Q: How would this scale to 10,000 contracts?

A: The architecture is designed for this. Swap ChromaDB for Azure AI Search — it handles millions of vectors with sub-second query time. Swap the local embedding model for Azure OpenAI embeddings service — batching is already implemented. Add an Azure Service Bus queue between the upload API and the ETL pipeline so ingestion is asynchronous and doesn't block the HTTP response. The LangGraph agent is stateless — run multiple instances behind Azure API Management.

Q: How do you handle GDPR data subject requests (right to erasure)?

A: Each chunk's metadata includes the source filename and MD5 checksum. To delete a contract: remove its chunks from ChromaDB by querying for `source_file == "contract.pdf"`, delete the raw file from Azure Blob Storage, and remove the registry entry. The LLM model weights are never updated — the contract text only ever lived in the vector store and blob storage. This is the key GDPR advantage of RAG over fine-tuning.

Q: What monitoring would you add for production?

A: Azure Monitor + Application Insights. Track: query latency (p50/p95/p99), retrieval hit rate (what fraction of queries get at least one chunk above threshold), LLM token usage (cost control), ingestion failures (by document type), compliance check throughput. Add distributed tracing across the LangGraph nodes so you can see which node is the bottleneck for a specific query. Run the evaluation harness weekly to catch model regressions after OpenAI model updates.

Q: What would you change if you had more time?

A: Four things in priority order. 1. Add `GET /api/files` listing endpoint so the sidebar persists across page refreshes. 2. Re-index ChromaDB with 1500-char chunks consistently (currently mixed 1000/1500 from different development phases). 3. Add streaming for the compliance check endpoint — currently it waits for the full response which can take 3–4 seconds for long contracts. 4. Add Azure Service Bus queue for async ingestion — currently the HTTP request blocks until ingestion completes, which takes 10–30 seconds for large PDFs.